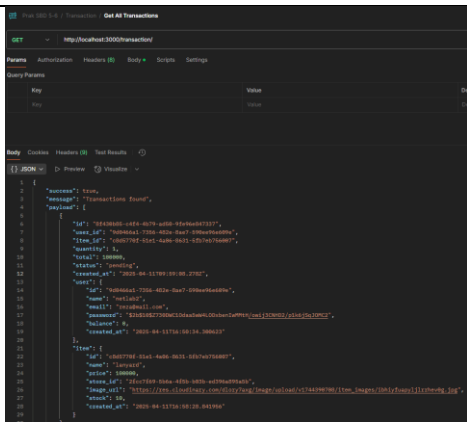
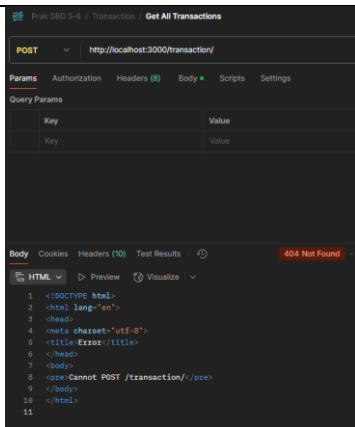


PRAKTIKUM SISTEM BASIS DATA

Nama	Muhammad Raditya Alif Nugroho	No. Modul	6
NPM	2306212745	Tipe	Tugas Tambahan

Screenshot Response

Method	Endpoint	Success	Failed
GET	/transaction		

Peningkatan Kode

Tipe	Dokumentasi	Deskripsi
Security	<pre> 3 const corsOptions = { 4 origin: "https://os.netlabdte.com", 5 methods: ["GET", "POST", "PUT", "DELETE"], 6 }; 7 app.use(cors(corsOptions)); 8 app.use(express.json()); 9 10 app.use('/store', require('./src/routes/store.route')); 11 app.use('/user', require('./src/routes/user.route')); 12 app.use('/item', require('./src/routes/item.route')); 13 app.use('/transaction', require('./src/routes/transaction.route')); </pre>	Implementasi CORS untuk membatasi domain yang dapat mengakses API, yaitu os.netlabdte.com

Security	<pre>const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+\$/; const passRegex = /^(?=.*\d)(?=.*[a-zA-Z]).{8,}\$/; exports.register = async (req, res) => { const { email, password, name } = req.query; if (!email !password !name) { return baseResponse(res, false, 400, "Email, password, and name are required", null); } if (!emailRegex.test(email)) { return baseResponse(res, false, 400, "Invalid email format", null); } if (!passRegex.test(password)) { return baseResponse(res, false, 400, "Invalid Password", null); } }</pre>	Implementasi Regex untuk memastikan nilai yang diinput sebagai email dan password memenuhi kriteria
Scalability	<pre>exports.getAllTransactions = async () => { try { const transactions = await db.query("SELECT * FROM transactions"); const userIds = transactions.rows.map((t) => t.user_id); const itemIds = transactions.rows.map((t) => t.item_id); const users = await db.query("SELECT * FROM users WHERE id = ANY(\$1)", [userIds]); const items = await db.query("SELECT * FROM items WHERE id = ANY(\$1)", [itemIds]); const userMap = new Map(users.rows.map((u) => [u.id, u])); const itemMap = new Map(items.rows.map((i) => [i.id, i])); return transactions.rows.map((transaction) => ({ ...transaction, user: userMap.get(transaction.user_id), item: itemMap.get(transaction.item_id), })); } catch (error) { console.error("Error fetching all transactions:", error); throw error; } };</pre>	Efisiensi query SQL dalam getAllTransaction agar lebih sederhana
Error Handling	<pre>exports.register = async (user) => { try { const hashedPassword = await bcrypt.hash(user.password, SALT_ROUNDS); const res = await db.query("INSERT INTO users (name, email, password, balance) VALUES (\$1, \$2, \$3, \$4) RETURNING *", [user.name, user.email, hashedPassword, 0]); return res.rows[0]; } catch (error) { console.error("Error executing query:", error); throw error; } };</pre>	Memastikan ada throw error pada seluruh error handling di repository. Dilakukan agar error dihandle oleh layer yang lebih tinggi